309. Best Time to Buy and Sell Stock with Cooldow

Dynamic Programming | LeetCode: https://leetcode.com/problems/best-time-to-b

Problem

- Unlimited transactions, one share at a time, and after selling you must wait one day before buying again. Maximise profit.

Intuition

- A state machine with three states per day:
- Hold (own a share), sold (just sold today, so cooling down tomorrow), rest (no share, free to buy).

Brute Force

- Recursion over buy/sell/skip choices per day: O(3^n), or memoised (day, holding) DP.

Optimal Approach

- Hold = -infinity, sold = 0, rest = 0.
- For price p: newHold = max(hold, rest - p); newSold = hold + p; newRest = max(rest, sold).
- Answer = max(sold, rest).

Worked Example

```
[1,2,3,0,2].

day0 p=1  hold=-1  sold=0   rest=0
p=2       hold=-1  sold=1   rest=0
p=3       hold=-1  sold=2   rest=1
p=0       hold=1   sold=-1  rest=2
p=2       hold=1   sold=3   rest=2   -> answer max(3,2)=3
```

Java Solution

```java
public class BestTimeToBuySellStockCooldown {
    public int maxProfit(int[] prices) {
        if (prices.length == 0) return 0;
        int hold = -prices[0], sold = 0, rest = 0;
        for (int i = 1; i < prices.length; i++) {
            int p = prices[i];
            int newHold = Math.max(hold, rest - p);
            int newSold = hold + p;
            int newRest = Math.max(rest, sold);
            hold = newHold;
            sold = newSold;
            rest = newRest;
        }
        return Math.max(sold, rest);
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Buying uses rest (not sold): that enforces the cooldown.
- Avoid Integer.MIN_VALUE + p overflow; start hold at -prices[0] instead.